

Отчет по заданию «А3. Точная функция $T(n)$ и порядок её роста»

Выполнил: студент Ошаров Александр группы БПИ248

Семинарист: Маров Руслан

Аннотация

В этом отчёте даётся анализ временной сложности функции `sum(int l, int r)`, реализующей запрос суммы на отрезке массива с предвычисленными суммами по блокам (sqrt-decomposition). В отчёте:

- Явно задаётся модель элементарных операций;
- Подсчитаны точные количества элементарных операций для каждой ветви условного оператора;
- Получено точное выражение T для одного запроса через числа обработанных одиночных элементов и полных блоков;
- Проведён строгий анализ худшего и лучшего случаев и доказано $T(n) = \Theta(\sqrt{n})$.

1. Постановка задачи

Имеется массив длины n , разбитый на блоки длины $k = \lfloor \sqrt{n} \rfloor$. Для каждого блока заранее вычислена сумма элементов и записана в массив `sqa`. Нужно проанализировать время выполнения функции:

```
int block = sqrt(n);
int sum(int l, int r) {
    int answer = 0;
    for (int i = l; i <= r; i++) {
        if (i % block != 0) {
            answer += a[i];
        } else if (i % block == 0) {
            answer += sqa[i / block];
        }
    }
    return answer;
}
```

Нужно: получить точную формулу $T(n)$ (включая учёт различного числа операций в ветвях), а также найти $f(n)$ такой, что $T(n) = \Theta(f(n))$.

2. Модель элементарных операций

Для точного подсчёта вводим следующую модель: каждая из перечисленных элементарных действий считается за 1 элементарную операцию:

- сравнение (например, $i \leq r$, $<=$, $!=$);
- арифметическая операция (+, -, *, /, %);
- доступ к элементу массива (чтение $a[i]$ или $\text{sq}[j]$);
- присваивание (включая комбинированные операторы вроде $\text{answer} += \dots$ рассматриваем как отдельную арифметическую операцию и отдельное присваивание);
- постинкремент $i++$ считается как одна арифметика ($i + 1$) и одно присваивание ($i = \dots$).

Такая модель детальна и соответствует требованию задачи учитывать разные ветви if/else.

3. Подсчёт операций по ветвям цикла

Обозначим $k = \lfloor \sqrt{n} \rfloor$, длина запроса $L = r - l + 1$. Введём обозначения:

- x — общее число итераций, в которых выполняется $\text{answer} += a[i++]$ (обработка поэлементно);
- y — число итераций, в которых выполняется $\text{answer} += \text{sq}[i / \text{block}]$; (обработка целого блока);
- $x = x_1 + x_2$, где
 x_1 — число одиночных-элементных итераций с $i \% k \neq 0$,
 x_2 — число одиночных-элементных итераций с $i \% k == 0$ (т. е. когда мы стоим на границе блока, но блок полностью не помещается в $[l, r]$). Замечание: $x_2 \in \{0, 1\}$ (объяснение ниже).

3.1. Подсчёт операций в одной итерации — три типа

Разберём три возможных типа итерации (включая все операции, выполняемые в теле и при проверках):

Тип А — $i \% k \neq 0$ (не в начале блока), выполняется ветвь $\text{answer} += a[i++]$:

- проверка цикла $i \leq r$: 1
- вычисление $i \% k$ и сравнение с 0: 1 (операция %) + 1 (сравнение) = 2
- исполнение $\text{answer} += a[i++]$: чтение $a[i]$ (1) + сложение (1) + присваивание answer (1) + инкремент i (1 арифметика + 1 присваивание = 2)

=> суммарно для ветви: 1 + 2 = 3 (для чтения/сложения/присваивания) + 2 ($i++$) = 5

Итого для итерации типа А: 1 (цикл) + 2 ($i \% k$ и сравнение) + 5 = 8 операций.

Тип В — $i \% k == 0$ и целый блок помещается (выполняется $answer += sq[a[i / block]]$; $i += k$):

- проверка цикла $i \leq r$: 1
- вычисление $i \% k$ и сравнение: $1 + 1 = 2$
- проверка $i + k - 1 \leq r$: вычисление $i + k$ (1) и -1 ещё (1) и сравнение (1) = 3
- исполнение ветви: деление $i / block$ (1) + чтение $sq[a[\dots]]$ (1) + сложение (1) + присваивание $answer$ (1) + выполнение $i += k$: сложение (1) + присваивание (1) => суммарно 6

Итого для итерации типа В: $1 + 2 + 3 + 6 = 12$ операций.

Тип С — $i \% k == 0$, но блок НЕ помещается (переход в `else` с $answer += a[i++]$):

- проверка цикла: 1
- $i \% k$ и сравнение: 2
- проверка $i + k - 1 \leq r$ (и она ложна): 3
- исполнение $answer += a[i++]$: как в типе А = 5

Итого для итерации типа С: $1 + 2 + 3 + 5 = 11$ операций.

4. Точная формула Т через x_1 , x_2 , y

Если в суммарно произошло x_1 итераций типа А, x_2 итераций типа С и y итераций типа В, то суммарное число элементарных операций (учитывая, что проверка цикла происходит ещё раз после выхода) равно:

$$T = 8 \cdot x_1 + 11 \cdot x_2 + 12 \cdot y + 1$$

Обоснование: мы суммируем 8 операций за каждую итерацию типа А, 11 за каждую типа С, 12 за каждую типа В и добавляем 1 — последнюю проверку условия цикла после выхода.

5. Связь с длиной отрезка L и простая форма выражения

Поскольку каждая итерация типа В продвигает индекс i на k элементов, а каждая итерация типа А/С — на 1, выполняется соотношение

$$L = r - l + 1 = x + k \cdot y = (x_1 + x_2) + k \cdot y.$$

Отсюда $x_1 = L - k \cdot y - x_2$. Подставляя в формулу для Т получаем (упрощённо):

$$T = 8 \cdot L - y \cdot (8k - 12) + 3 \cdot x_2 + 1$$

Это удобная интерпретация: если бы мы всегда обрабатывали элементы по одному, стоило бы $8 \cdot L$ операций.

Каждый раз, когда мы вместо k одиночных шагов делаем одну операцию по блоку

(type B), мы экономим $8k - 12$ операций (экономия положительна при $k \geq 2$). Погрешность связана с возможным единичным случаем x_2 (когда на границе есть неполный блок).

6. Оценка худшего случая $T(n)$ при заданном n

Интерпретируем $T(n)$ как максимально возможное время выполнения одного запроса при фиксированном n (т. е. берём максимум по всем возможным l, r). Покажем, что этот максимум имеет порядок $\sqrt{n}$.

1) Верхняя оценка ($O(\sqrt{n})$).

Заметим общие ограничения: $x = x_1 + x_2 \leq 2(k - 1)$ (может быть не более двух неполных блоков: левый и правый, каждый даёт $\leq k - 1$ одиночных обработок). Также $x_2 \leq 1$.

Используя формулу $T = 8x + 12y + 3x_2 + 1$ и оценку $x \leq 2k - 2$, $x_2 \leq 1$, $y \leq n/k$, получаем:

$$T \leq 8(2k - 2) + 12 \cdot y + 3 \cdot 1 + 1 = 16k - 16 + 12y + 4 = 12y + 16k - 12.$$

Подставляя $y \leq n/k \leq k$ (поскольку $k \approx \sqrt{n}$), получаем

$$T \leq 12k + 16k - 12 = 28k - 12 = O(k) = O(\sqrt{n}).$$

Таким образом существует константа c_2 (например, $c_2 = 28$), и n_0 , такие что для всех $n \geq n_0$: $T(n) \leq c_2 \cdot \sqrt{n}$.

2) Нижняя оценка ($\Omega(\sqrt{n})$).

Для получения нижней оценки приведём пример отрезка, дающего $\Omega(\sqrt{n})$ операций. Возьмём отрезок длины $L = k - 1$ (внутри одного блока) — тогда $y = 0$, $x = k - 1$, x_2 может быть 0 или 1.

Тогда по формуле $T \geq 8 \cdot (k - 1) \geq 8 \cdot (\sqrt{n} - 1) - \text{const}$, т. е. $T = \Omega(k) = \Omega(\sqrt{n})$.

Следовательно, существует $c_1 > 0$ и n_0 , такие что для всех $n \geq n_0$ $T(n) \geq c_1 \cdot \sqrt{n}$ (например, можно взять $c_1 = 4$ при достаточно больших n).

7. Вывод — доказательство $T(n) = \Theta(\sqrt{n})$

Комбинируя оценки: найдутся положительные константы c_1, c_2 и n_0 такие, что для всех $n \geq n_0$:

$$c_1 \cdot \sqrt{n} \leq T(n) \leq c_2 \cdot \sqrt{n}.$$

Следовательно $T(n) = \Theta(\sqrt{n})$. Интуитивно: либо запрос состоит почти целиком из двух

неполных блоков (тогда обрабатывается $O(k)$ отдельных элементов), либо содержит много полных блоков (и их число $\approx n/k = k$), в любом случае число шагов пропорционально $k = \Theta(\sqrt{n})$.

8. Иллюстрация (небольшой числовой пример)

Пусть $n = 10\,000 \Rightarrow k = \lfloor \sqrt{10000} \rfloor = 100$.

- Если отрезок не содержит целых блоков и длина $L = 2k - 2 = 198$, то примерная оценка даёт
 $T \approx 8 \cdot 198 + 11 \cdot 1 + 1 \approx 1584 + 12 = 1596$ операций.
- Если отрезок большой и покрывает почти весь массив, то $y \approx n/k = 100$, и $T \approx 12 \cdot 100 + \text{const} = 1200$.

Оба числа — порядка $k = 100$, что подтверждает $\Theta(\sqrt{n})$ -оценку.

9. Заключение и замечания по отчёту

- В отчёте явным образом задана модель элементарных операций и подсчитаны операции для каждой ветви условного оператора; получена точная формула $T = 8x_1 + 11x_2 + 12y + 1$.
- Получено удобное представление $T = 8L - y(8k - 12) + 3x_2 + 1$, откуда видно экономию при использовании предвычисленных сумм по блокам.
- В худшем и лучшем случаях время всё равно пропорционально $k = \lfloor \sqrt{n} \rfloor$, поэтому $T(n) = \Theta(\sqrt{n})$.